

山东大学
SHANDONG UNIVERSITY

《计算机组成与设计》课程设计报告

题目：简单模型机设计（微程序实现）

姓名：曲东芸
学号： 202300130185
班级：计科 23.1

2025 年12 月12 日

目录

1 总述..... 1
1.1 课程设计目的..... 1
1.2 课程设计工具..... 1
1.3 课程设计方法..... 1
2 微程序控制下的指令系统..... 2
2.1 指令格式..... 2
2.2 基本字长..... 2
2.3 指令类型..... 2
2.4 寻址方式..... 3
2.5 实现指令组..... 3
3 总体结构与数据通路..... 6
3.1 总体结构框图..... 6
3.2 各个部件功能及原理图..... 6
4 微程序-CU 10
4.1 CU 的实现及其部件原理图 10
4.2 指令的执行流程..... 13
4.3 微指令格式与 ROM 微程序..... 15
4.4 RAM 内的应用程序..... 19
5 课程设计总结..... 20
6.附录..... 21
附录 1 数据通路图..... 21
附录 2 微程序流程图..... 21
附录 3 微程序..... 22
附录 4 控制信号列表..... 23
附录 5 控制信号逻辑表达式..... 24

1 总述

1.1 课程设计目的

通过计算机组成原理的课程设计，进一步深入理解计算机组成的原理，学会运用计算机底层基础知识，充分提高动手能力，巩固对于 CPU 的理解，详细了解微程序和硬布线的设计思路以及其他基础知识，从而提高学生各方面认知、实践能力。

1.2 课程设计工具

软件： vivado、Windows 11

1.3 课程设计方法

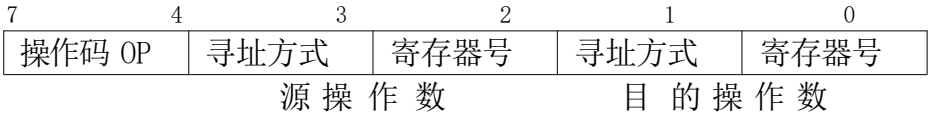
课程设计:微程序实现的模型机内核。通过对于μPC，μIR,CROM 等微程序部件的设计，来实现微程序控制整个 CPU 配合 主存 RAM 进行各种指令的执行。

2 微程序控制下的指令系统

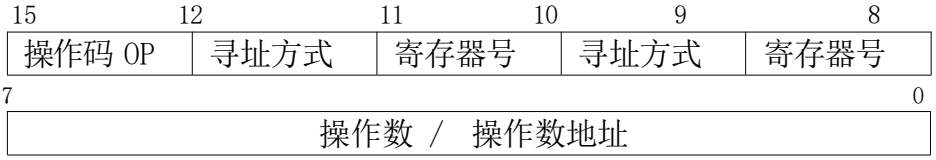
2.1 指令格式

指令格式有单字长指令和双字长指令，在双字长指令之中，第二字节通常是操作数的值或者是操作数的地址。

单字长指令格式如下：



双字长指令格式如下：



对于执行功能相似的指令，其前 4 位相同，形成指令的操作码。其余指令， 由于指令的总条数超过 16 条，并且部分指令的微指令较多，所以部分指令的 操作码到达 8 位（也就等于其完整指令编码）。

2.2 基本字长

主存 RAM 的基本字长为 1 byte 即 8 位，地址线 8 条，微程序存储器 CROM 的基本微指令字长为 32 位。

2.3 指令类型

1) 模型机有单操作数指令、双操作数指令、无操作数指令
2) 操作码有 4 位，最多可以定义 16 条不同的指令
3) 无操作数指令有： Fetch 取指指令，HALT 停机指令
4) 单操作数指令有： LD，STA 取数存数指令， DEC 自减指令 JMP 无条件跳转等
5) 双操作数指令有： JPZ 有条件跳转指令等

2.4 寻址方式

1）寻址方式位为 00 为立即数寻址，操作数在指令的下一个单元

2）寻址方式位为 01 为寄存器寻址，操作数在寄存器中，寄存器号为 0 代表操作数在 R0 寄存器中，寄存器号为 1 代表操作数在 R1 寄存器中

3）寻址方式位为 10 为直接寻址， 操作数的地址在指令的下一个单元 4）寻址方式位为 11 为寄存器间接寻址， 操作数的地址在寄存器里面

指令寻址方式表

寻址方式	**指令名称**
立即数寻址	MOV0,MOV1,FAC
直接寻址	STA0,STA1,LD0,LD1 PUSH,JMP,JPZ,JPNZ
寄存器寻址	ADD,INC,DEX,SUB,MUL,DIV, MOD,SWP,AND,OR,NOT,XOR
寄存器间接寻址	JMP[R0]

2.5 实现指令组

1.实现功能：

在微程序实现的模型机的指令系统中，所有的指令，都是对应的微程序段，在 CROM 中的入口地址，例如 MOV 指令为 0X10H, 则说明执行 MOV 的微程序段，在 CROM 中，第一条微指令的地址是 0X10H, 其余指令也是如此。

（1）MOV R0,#0X55AA

功能：将 16 位立即数 0x55aa 传送到寄存器 R0

执行过程：取指令操作码；读取下一个内存单元中的立即数；将立即数传送到 R0 寄存器

（2）LDR R1,0XA0

功能：从内存地址 0xA0 处读取数据，存入寄存器 R1

执行过程：；取指令操作码；读取下一个内存单元中的地址 0xA0；以该地址访问内存读取数据；将数据存入 R1 寄存器

（3）ADD R0,R1

功能：将 R0 和 R1 的内容相加，结果存入 R0

执行过程：R0 送 X 寄存器；ALU 执行 X+R1 操作，结果暂存 Z 寄存器；将 Z 寄存器内容传送到 R0

（4）MOV R2,#0XFA

功能：将 8 位立即数 0xFA 传送到寄存器 R2

（5）STR R0,[R2]

功能：将 R0 的内容存储到内存中，内存地址由 R2 的内容指定

执行过程：将 R2 的内容作为地址送 MAR；将 R0 的内容送 MDR；执行内存写操作，将 MDR 内容写入 MAR 指定的内存单元

（6）HALT

功能：停止处理器执行

执行过程：设置 G 标志为 1，使处理器停止

2. 指令格式：

所有指令均为 16 位，从二进制高位到低位拆分，优先确定操作码字段（需覆盖 6 条指令，至少 4 位），剩余位数为地址码字段（按指令功能细分为寄存器号、立即数、内存地址等子字段）；

通用寄存器 R0~R2 的编码为 3 位

（1）MOV R0,#0X55AA

机器指令 0X0500，转成二进制是 0000 0101 0000 0000

为 32 位（4 字节）立即数 MOV 指令：前 2 字节（0x0500）为 “操作码 + 目标寄存器 + 立即数高位”，后 2 字节为立即数低位（0x55aa）

操作码二进制编码 0000，地址码是立即数寻址

操作码 0000：触发微代码 “立即数读取流程”（PC→MAR→MDR 读立即数）；

目标寄存器号 000（R0）：对应微操作 MDR→R0（微地址 018）；

立即数 0x55aa：分 2 字节存储（高位 0x55，低位 0xaa），通过微操作加载到 MDR 后写入 R0

（2）LDR R1,0XA0

机器指令 0x0938，转化成二进制是 0000 1001 0011 1000，操作码 0000，四位，地址码 12 位

操作码 0000 1001（扩展为 8 位子操作码）：触发 “直接寻址读流程”（微地址 020-025）；

目标寄存器号 001（R1）：对应微操作 MDR→R1（微地址 024）；

直接地址 0xA0（9 位编码 1001 11000）：通过微操作 MDR→MAR（微地址 022）指向内存 0xA0 单元，读取数据后写入 R1，匹配 “从 0xA0 读数据到 R1” 的

功能。

（3）ADD R0,R1

机器指令 0x1109,转化成二进制 0001 0001 0000 1001，操作码 8 位：00010001，标识 “寄存器加法指令”，对应微代码入口 030（微操作：R0→X→ALU+R1→Z→R0），地址码 8 位

操作码 0001 0001：触发 “寄存器加法流程”（微地址 030-033）；

被加数寄存器 000（R0）：对应微操作 R0→X（微地址 030，ALU 输入 1）；

加数寄存器 001（R1）：对应微操作 X+R1→Z（微地址 031，ALU 输入 2）；

结果回写 R0：对应微操作 Z→R0（微地址 032），完全匹配 “R0=R0+R1” 的功能

（4）MOV R2,#0XFA

机器指令是 0x0540,转成二进制是 0000 0101 0100 0000，操作码四位，与 MOV R0,#0x55aa 同格式，32 位指令

操作码 0000：同属立即数 MOV 类型，对应微地址 010；

目标寄存器号 010（R2）：触发微操作 MDR→R2（微地址 01C）；

立即数 0xFA（8 位，扩展为 16 位 0x00FA）：通过微操作读取后写入 R2，匹配 “初始化 R2 为 0xFA” 的功能。

（5）STR R0,[R2]

机器码 0x0D12，对应二进制 0000 1101 0001 0010，操作码八位：0000 1101，地址码 8 位

操作码 0000 1101：触发 “间接寻址写流程”（微地址 040-043）；

基址寄存器 010（R2）：对应微操作 R2→MAR（微地址 040，指向目标内存地址）；

源数据寄存器 000（R0）：对应微操作 R0→MDR（微地址 041，待存储数据）；

内存写操作：对应微操作 MDR→M(MAR)（微地址 042），匹配 “将 R0 数据写入 R2 指向的内存” 的功能。

（6）HALT

机器码 0x1400,对应二进制 0001 0100 0000 0000，操作码 16 位，唯一标识 “停机指令”，无地址码，对应微代码入口 050（微操作：G=1，CPU 停机）

3 总体结构与数据通路

3.1 总体结构框图

CPU 内部结构采用单总线方式

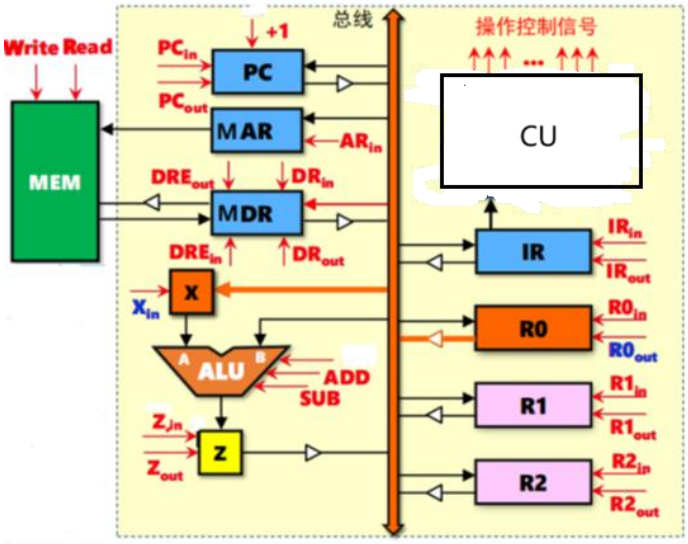
控制信号：

IRout、PCout、…R1out 控制三态门将寄存器值输出到总线

IRin、PCin、…R1in　控制寄存器使能端将总线数据锁存（边沿触发）

ADD、SUB　运算控制信号

Write、Read　内存读写控制信号

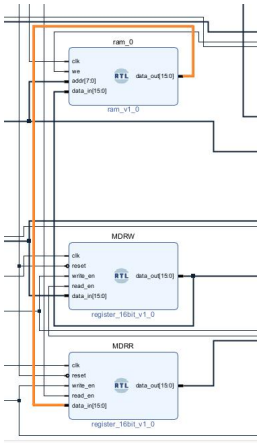


3.2 各个部件功能及原理图

1. 主存 MEM

功能：按字编址（字长 8 位，容量 64KW），存放机器指令和数据；通过 MAR 接收地址、MDR 接收 / 发送数据，响应微程序控制器的 READ/WRITE 控制信号；操作完成后向 CPU 发送 MOC（主存操作完成）信号，确保同步

设计原理图：



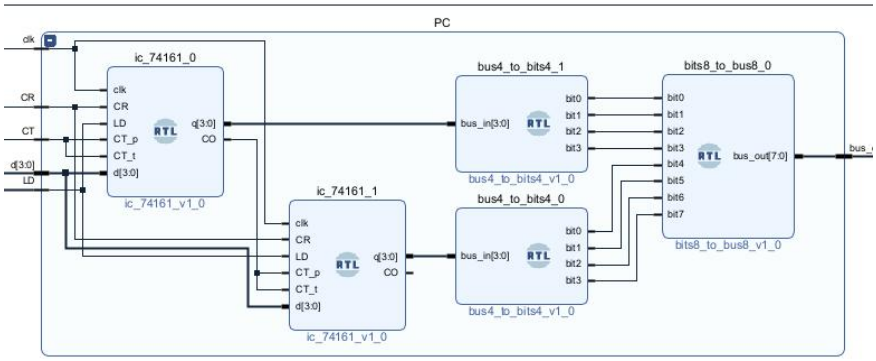
RAM 里存的是地址：

```
@0
0500
55aa
0938
00A0
1109
0540
00FA
0D12
1400
@A0
aa55
```

2. PC 程序计数器

功能：存储下一条要执行的指令在主存中的地址，取指后自动 PC+1（更新为下一条指令地址）；跳转指令时接收新地址（来自总线，覆盖原地址）；复位时清零（从主存 0 地址开始取指）

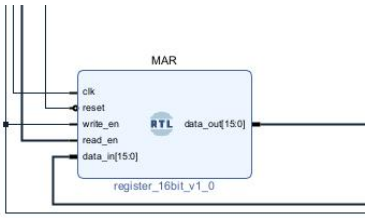
原理图：



3. 地址寄存器（MAR）

功能：暂存要访问的主存单元地址（来自 PC 或总线），避免地址信号冲突；增强地址信号驱动能力，直接输出到主存地址总线；系统复位时同步清零，确保初始地址为 0。

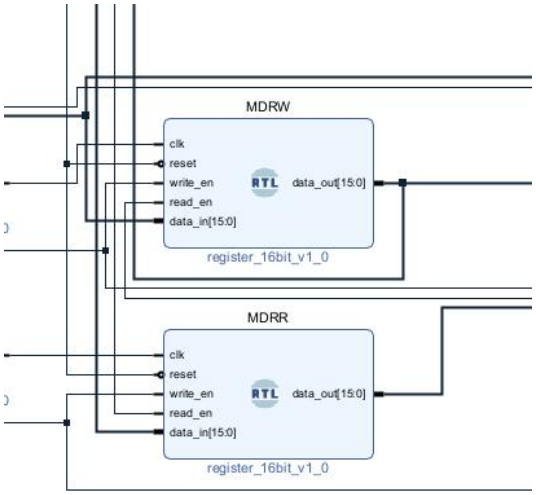
原理图：



4. 数据寄存器（MDR）

功能：作为 CPU 与主存之间的数据缓冲，解决 CPU 与主存速度不匹配问题；

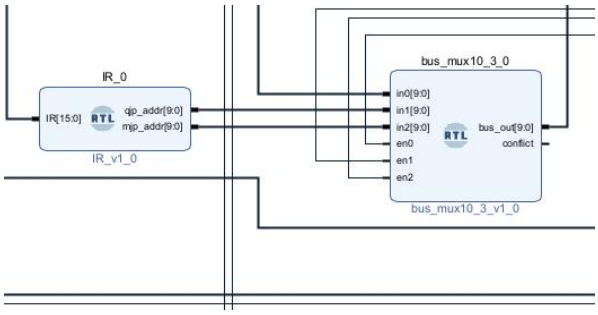
原理图：

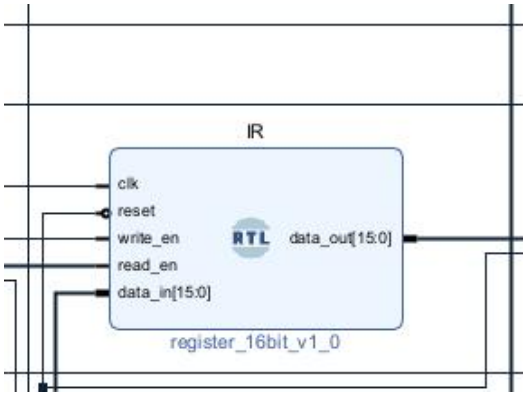


5. 指令寄存器（IR）

功能：存储当前正在执行的机器指令（来自 MDR），将 8 位指令拆分为操作码（高 4 位）和地址码（低 4 位），操作码送微程序控制器译码，地址码送地址生成逻辑；在指令执行周期内保持指令不变，确保控制信号持续有效。

原理图：

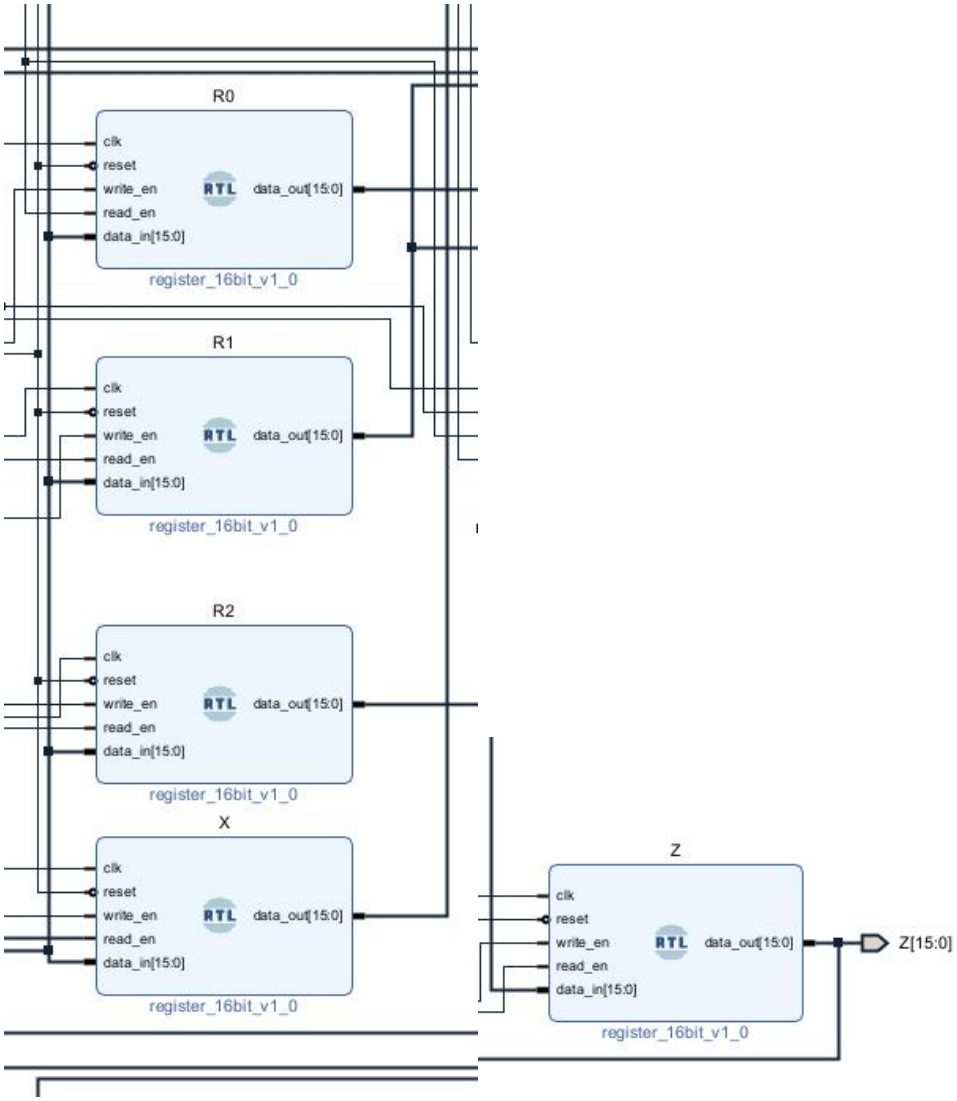




6. 通用寄存器 R0 R1 R2

功能：3 个 8 位可编程寄存器，供用户存放操作数、中间结果或最终结果；通过系统数据总线与 ALU、MDR 交互，支持 R→R、R→M、M→R 操作；仅在微控制器发出对应寄存器的 CP_Rn 信号时更新数据，确保数据一致性。

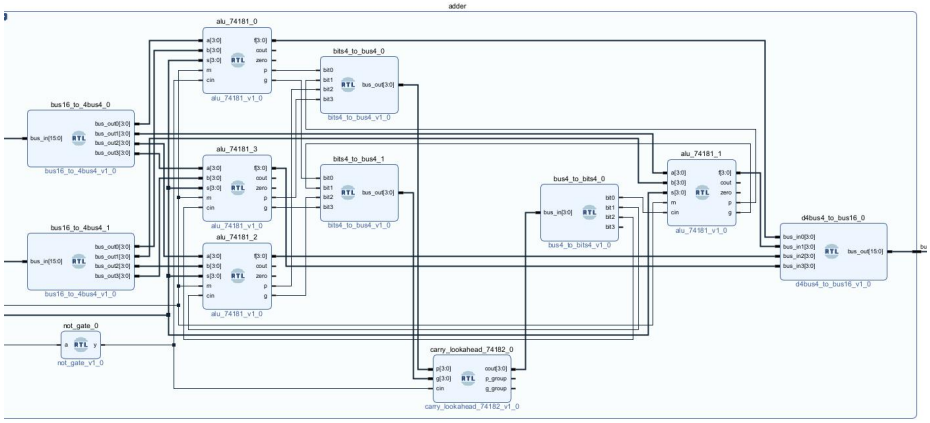
原理图：



7. ALU

功能：ALU 核心：执行算术运算（加法、减法）和逻辑运算（与、或），本模型机以加法为核心（支持 PC+1、R0+R1）；

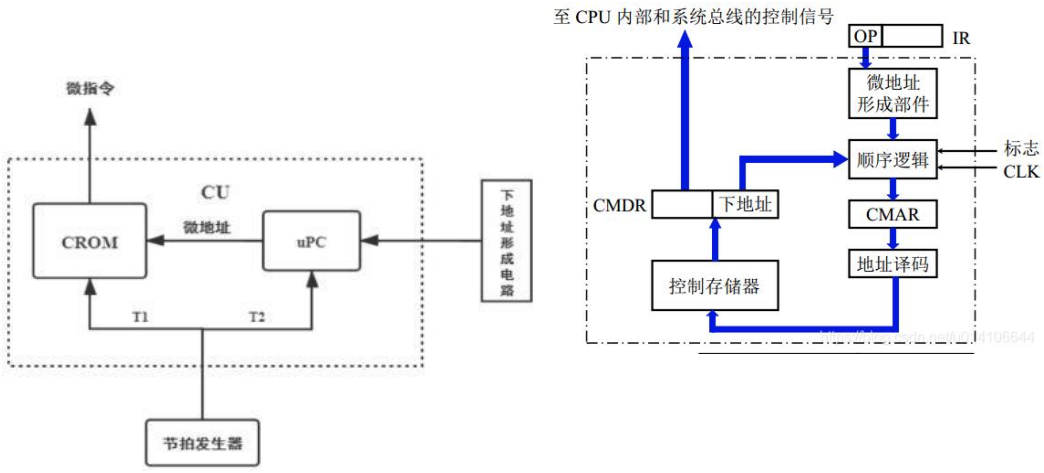
原理图：



4 微程序-CU

4.1 CU 的实现及其部件原理图

1. CU 框图：



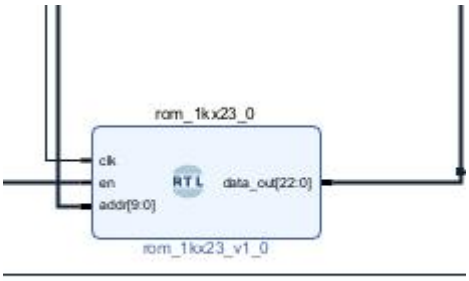
微程序实现的控制部件 CU 的总体结构图如上图所示，包括储存微指令的 CROM，负责产生微地址的 μPC 等。

2. 控制存储器 CROM

功能：存储所有微程序（取指微程序 + 6 条指令的执行微程序），按微地址输出 23 位微指令

CROM 读取微指令，需要等到节拍的上升沿处，才能将微指令读出，需 要为 CROM 分配节拍。由于 μPC 的初始值为 00H，因此，将节拍发生器 的第一个节拍安排给 CROM，上升沿到来时，可以直接读出位于 CROM 中的 00H 地址的微指令。

原理图：



下面是 ROM.V 的内容

```
// μIR12-μIR11: PC计数器 = 00 = 2'b00
// μIR10: Ein = 0 = 1'b0
// μIR9: DREin = 0 = 1'b0
// μIR8-μIR6: 输入in分配 = 110 = 3'b110
// μIR5-μIR4: RAM操作 = 00 = 2'b00
// μIR3: G = 0 = 1'b0
// μIR2-μIR0: 转移方式 = 001 = 3'b001
// 组合：000000_0_111_00_0_0_110_00_0_001 = 23'b0000000_11100_0_0_11000_0_001
// 十六进制：0x00E181
rom_memory[10'h000] = 23'h00E181;  // 地址0

// 第2行：M(MAR)->MDR PC+1->PC，微地址001（十进制1）
// 000000_0_000_01_0_1_000_01_0_001 = 23'b0000000_00001_0_1_00001_0_001
// 十六进制：0x00A11
rom_memory[10'h001] = 23'h000A11;  // 地址1

// 第3行：MDR->IR，微地址002（十进制2）
// 000000_0_110_00_0_0_001_00_0_001 = 23'b0000000_11000_0_0_00100_0_001
// 十六进制：0x00C041
rom_memory[10'h002] = 23'h00C041;  // 地址2

// 第4行：QJP，微地址003（十进制3）
// 000000_0_000_00_0_0_000_00_0_011 = 23'b0000000_00000_0_0_00000_0_011
// 十六进制：0x000003
rom_memory[10'h003] = 23'h000003;  // 地址3

// 第5行：PC->MAR，微地址010（十进制16）
// 与地址0相同
rom_memory[10'h010] = 23'h00E181;  // 地址16

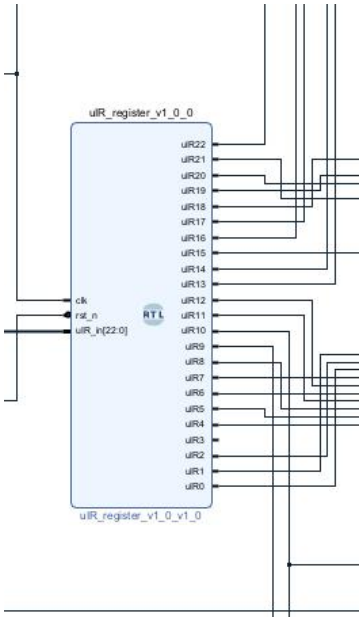
// 第6行：M(MAR)->MDR PC+1->PC，微地址011（十进制17）
// 000000_0_000_01_0_1_000_01_0_001 = 23'b0000000_00001_0_1_00001_0_001
// 十六进制：0x00A11，但表格显示为0A 14，所以用0x000A14
rom_memory[10'h011] = 23'h000A11;  // 地址17

// 第7行：MJP，微地址012（十进制18）
```

3.微指令寄存器 uIR

功能：寄存从 ROM 读出的 23 位微指令，将其拆分为操作控制字段（驱动 CPU 部件）和顺序控制字段（生成下地址）；

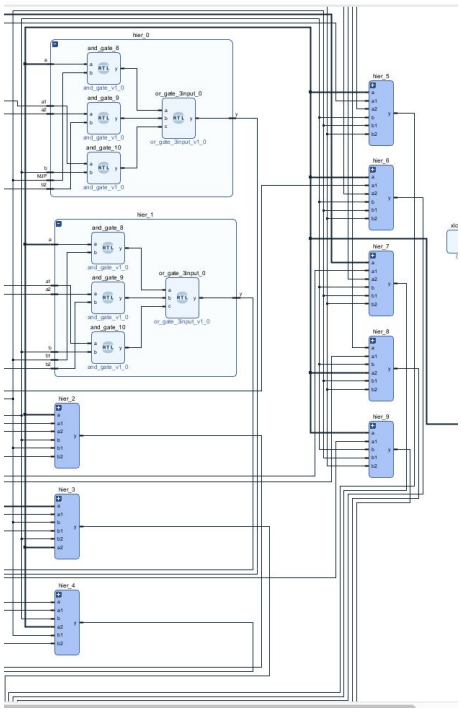
原理图：



4.多路选择器-微地址形成电路

功能：根据μIR的顺序控制字段（Q2:Q0）、指令操作码译码结果、RAM的MOC信号，生成下一条微指令的地址（更新微地址寄存器 μAR）；

原理图：

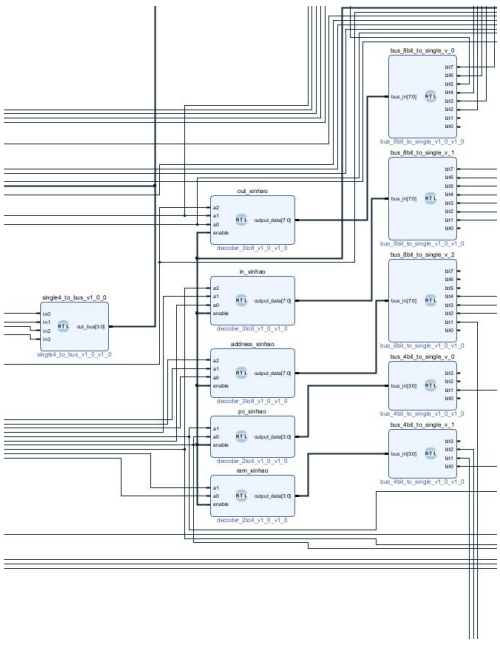


5.控制信号译码器

功能：将 μIR 的操作控制字段（Q22:Q3）拆分为 CPU 各部件的具体控制信号（如PC_inc、RAM_we、RO_en）

38 译码器来根据输入的微指令发出对应的信号，例如片选信号、 打入信号、下地址形成方式信号等等

原理图：



6.微程序计数器 UPC

功能：存储当前正在执行的微指令在控制存储器（ROM）中的地址，并根据微指令的 “顺序控制字段” 更新地址，驱动微程序按序执行

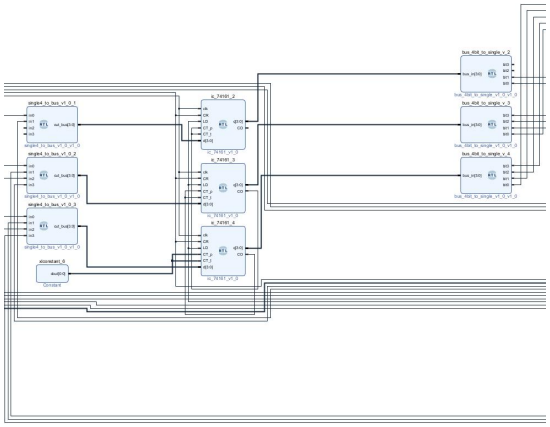
产生读取 CROM 时的微地址，充当微地址形成电路。微地址的产 生方式受到下地址产生电路的控制，共有 3 种产生方式:

（1） 顺序产生。此时 μPC 会执行 μPC+1, 产生微地址。该方式主要用于 提供正在执行的指令，对应的微程序内的微地址。

（2）按照正在执行的微指令的第 8-15 位产生微地址，此时 μPC 会执行 置数操作，将 μPC 的值更改为当前微指令 8-15 位的值。这种形成 方式用于在一段微程序内进行跳转。

（3）按照 IR 提供的 8 位地址无条件转移，此时 nPC 会执行置数操作， 将″PC 的值更改为 IR 的值。在指令系统部分已经说明，指令实际 是执行指令的微程序的入口地址。

原理图：



4.2 指令的执行流程

所有指令共享取指周期（4 个节拍），执行周期因指令功能不同分为 2~5 个节拍，节拍与 ROM 微地址一一对应（微地址十进制 = 节拍序号）。

1. 取值周期：

节拍	微地址	微指令	微操作
T0	0	0x00E181	PC->MAR
T1	1	0x000A11	M(MAR)->MDR, PC+1->PC
T2	2	0X00C041	MDR->IR
T3	3	0X000003	QJP

2.各指令的执行周期：

（1）MOV R0,#0x55aa（立即数传送，微地址 16~18、24~25，5 个节拍）

节拍	微地址	微指令	微操作
T4	16	0x00E181	PC→MAR
T5	17	0x000A11	M(MAR)→MDR，PC+1→PC
T6	18	0x000004	MJP（无条件转移）
T7	24	0x00C081	MDR→R0
T8	25	0x000002	JP（返回取指）

(2) LDR R1,0xA0（直接寻址读，微地址 32~37，6 个节拍）

节拍	微地址	微指令	微操作
T4	32	0x00E181	PC→MAR
T5	33	0x000A11	M(MAR)→MDR，PC+1→PC
T6	34	0x00C181	MDR→MAR
T7	35	0x000211	M(MAR)→MDR
T8	36	0x00C0C1	MDR→R1
T9	37	0x000002	JP（返回取指）

(3) ADD R0,R1（寄存器加法，微地址 48~51，4 个节拍）

节拍	微地址	微指令	微操作
T4	48	0x004141	R0→X（ALU 输入寄存器）
T5	49	0x246401	X+R1→Z（ALU 输出寄存器）
T6	50	0x00A081	Z→R0
T7	51	0x000002	JP（返回取指）

(4) MOV R2,#0xFA（立即数传送，微地址 16~18、28~29，5 个节拍）

节拍	微地址	微指令	微操作
T4	16	0x00E181	PC→MAR
T5	17	0x000A11	M(MAR)→MDR，PC+1→PC
T6	18	0x000004	MJP（无条件转移）
T7	28	0x00C101	MDR→R2
T8	29	0x000002	JP（返回取指）

（5）STR R0,[R2]（寄存器间接写，微地址 64~67，4 个节拍）

节拍	微地址	微指令	微操作
T4	64	0x008181	R2→MAR
T5	65	0x0041C1	R0→MDR
T6	66	0x010021	MDR→M(MAR)
T7	67	0x000002	JP（返回取指）

（6）HALT（停机指令，微地址 80，1 个节拍）

节拍	微地址	微指令	微操作
T4	80	0x000008	R2→MARG=1（停机控制信号）

4.3 微指令格式与 ROM 微程序

1.微指令格式：

基于 ROM 初始化时的字段定义，23 位微指令分为操作控制字段（20 位） 和顺序控制字段（3 位）

字段名称	位范围（μIR [22:0]）	位数	编码含义	微命令示例
ALU 控制字段	22~17	6	M（模式）+ S3~S0（运算选择）+ C0（进位输入）- M=0：算术模式；M=1：逻辑模式- S3~S0=0011：加法；0101：减法	000000（无 ALU 操作）、010011（加法，C0=1）
DREout	16	1	数据总线输出使能：1 = 允许输出；0 = 禁止	0（禁止）、1（允许 MDR 输出到总线）
输出分配字段	15~13	3	选择输出到总线的部件：- 000：无；001：PC；010：R0；011：R1；100：R2；110：MDR；111：Z	110（MDR 输出）、010（R0 输出）
PC 控制字段	12~11	2	PC 操作：00 = 无；01=PC+1；10=PC 清零；11=PC 加载总线数据	00(无)、01(PC+1)
Zin	10	1	ALU 输出寄存器 Z 的写使能：1 = 允许写入；0 = 禁止	0（禁止）、1（允许 X+R1→Z）
DREin	9	1	数据总线输入使能：1 = 允许输入；0 = 禁止	0（禁止）、1（允许总线数据写入 MDR）
输入分配字段	8~6	3	选择从总线输入的部件：- 000：无；001：IR；010：R0；011：R1；100：R2；110：MAR；111：MDR	110（MAR 输入）、010（R0 输入）
RAM 操作字段	5~4	2	RAM 读写控制：00 = 无；01 = 读（RAM_rd=1）；10 = 写（RAM_we=1）；11 = 保留	00(无)、01(读)、10（写）

字段名称	位范围（μIR [22:0]）	位数	编码含义	微命令示例
G（停机字段）	3	1	停机控制：1=CPU 停机；0 = 正常运行	0（正常）、1（停机）
顺序控制字段	2~0	3	下地址转移方式:- 000:保留;001:顺序（当前 +1）；010：返回取指（000）；011：条件转移；100：无条件转移	

2. ROM 内的微程序及地址

微地址（十进制）	微地址（十六进制）	微指令（23 位十六进制）	对应字段拆分（关键部分）	微操作	所属周期
0	0x000	0x00E181	输出分配 = 111（MDR）、输入分配 = 110（MAR）、转移 = 001	PC→MAR	取指 T0
1	0x001	0x000A11	PC 控制 = 01（PC+1）、RAM 操作 = 01（读）、转移 = 001	M(MAR) → MDR，PC+1→PC	取指 T1
2	0x002	0x00C041	输出分配 = 110（MDR）、输入分配 = 001（IR）、转移 = 001	MDR→IR	取指 T2
3	0x003	0x000003	转移 = 011（条件）	QJP（按操作码跳转）	取指 T3
16	0x010	0x00E181	输出分配 = 111（MDR）、输入分配 = 110（MAR）、转移 = 001	PC→MAR	MOV 执行 T4
17	0x011	0x000A11	PC 控制 = 01（PC+1）、RAM 操作 = 01（读）、转移 = 001	M(MAR) → MDR，PC+1→PC	MOV 执行 T5

微地址（十进制）	微地址（十六进制）	微指令（23位十六进制）	对应字段拆分（关键部分）	微操作	所属周期
18	0x012	0x000004	转移 =100（无条件）	MJP（跳微地址24/28）	MOV 执行 T6
24	0x018	0x00C081	输出分配 = 110（MDR）、输入分配 = 010（R0）、转移 =001	MDR→R0	MOV R0 执行 T7
25	0x019	0x000002	转移 = 010（返回取指）	JP（回 000）	MOV R0 执行 T8
28	0x01C	0x00C101	输出分配 = 110（MDR）、输入分配 = 100（R2）、转移 =001	MDR→R2	MOV R2 执行 T7
29	0x01D	0x000002	转移 = 010（返回取指）	JP（回 000）	MOV R2 执行 T8
32	0x020	0x00E181	输出分配 = 111（MDR）、输入分配 = 110（MAR）、转移 = 001	PC→MAR	LDR 执行 T4
33	0x021	0x000A11	PC 控制 = 01（PC+1）、RAM 操作 = 01（读）、转移 = 001	M(MAR) → MDR，PC+1→PC	LDR 执行 T5
34	0x022	0x00C181	输出分配 = 110（MDR）、输入分配 = 110（MAR）、转移 = 001	MDR→MAR	LDR 执行 T6
35	0x023	0x000211	RAM 操作 =01（读）、转移 = 001	M(MAR)→MDR	LDR 执行 T7
36	0x024	0x00C0C1	输出分配 = 110（MDR）、输入分配 =	MDR→R1	LDR 执行

微地址（十进制）	微地址（十六进制）	微指令（23位十六进制）	对应字段拆分（关键部分）	微操作	所属周期
			011(R1)、转移 =001		T8
37	0x025	0x000002	转移 = 010（返回取指）	JP（回 000）	LDR 执行 T9
48	0x030	0x004141	输出分配 = 010（R0）、输入分配 = 101(X)、转移 =001	R0→X	ADD 执行 T4
49	0x031	0x246401	ALU 控制 = 010011（加法）、Zin=1、转移 = 001	X+R1→Z	ADD 执行 T5
50	0x032	0x00A081	输出分配 =101(Z)、输入分配 = 010（R0）、转移 = 001	Z→R0	ADD 执行 T6
51	0x033	0x000002	转移 = 010（返回取指）	JP（回 000）	ADD 执行 T7
64	0x040	0x008181	输出分配 = 100（R2）、输入分配 = 110（MAR）、转移 = 001	R2→MAR	STR 执行 T4
65	0x041	0x0041C1	输出分配 = 010（R0）、输入分配 = 111（MDR）、转移 = 001	R0→MDR	STR 执行 T5
66	0x042	0x010021	RAM 操作 =10(写)、转移 = 001	MDR→M(MAR)	STR 执行 T6
67	0x043	0x000002	转移 = 010（返回取指）	JP（回 000）	STR 执行 T7
80	0x050	0x000008	G=1（停机）、转移 = 000	G=1（CPU 停机）	HALT 执行

微地址（十进制）	微地址（十六进制）	微指令（23位十六进制）	对应字段拆分（关键部分）	微操作	所属周期
					T4

4.4 RAM 的应用程序

RAM 为 256×16 位，地址 0~8 存储应用程序，对应 6 条指令的汇编与机器语言.

RAM 地址（十进制）	机器语言（16位十六进制）	汇编指令	指令类型	说明
0	0x0500	MOV R0,#0x55aa	立即数传送	指令操作码：0x05，目标寄存器 R0，立即数存于 RAM 地址 1
1	0x55aa	（立即数）	-	MOV R0 指令的 16 位立即数，与地址 0 的指令构成完整指令
2	0x0938	LDR R1,0xA0	直接寻址读	指令操作码：0x09，目标寄存器 R1，直接地址 0xA0 存于 RAM 地址 3
3	0x00A0	（直接地址）	-	LDR R1 指令的 16 位直接地址，与地址 2 的指令构成完整指令
4	0x0D09	ADD R0,R1	寄存器加法	指令操作码：0x0D，被加数寄存器 R0，加数寄存器 R1
5	0x0540	MOV R2,#0xFA	立即数传送	指令操作码：0x05，目标寄存器 R2，立即数存于 RAM 地址 6
6	0x00FA	（立即数）	-	MOV R2 指令的 16 位立即数（0xFA），与地址 5 的指令构成完整指令
7	0x1112	STR R0,[R2]	寄存器间接写	指令操作码：0x11，源寄存器 R0，基址寄存器 R2（地址 0xFA）

RAM 地址（十进制）	机器语言（16位十六进制）	汇编指令	指令类型	说明
8	0x1400	HALT	停机指令	指令操作码：0x14，无地址码，触发 CPU 停机

RAM 地址 0~1：执行 MOV R0,#0x55aa，R0 被初始化为 0x55aa；
RAM 地址 2~3：执行 LDR R1,0xA0，从 RAM 0xA0 单元读取数据到 R1；
RAM 地址 4：执行 ADD R0,R1，R0 = R0 + R1，完成加法运算；
RAM 地址 5~6：执行 MOV R2,#0xFA，R2 被初始化为 0xFA（目标 RAM 地址）；
RAM 地址 7：执行 STR R0,[R2]，将 R0 的加法结果写入 RAM 0xFA 单元；
RAM 地址 8：执行 HALT，CPU 停机，程序结束。

5. 课程设计总结

通过这次的课设，我从电路原理的角度， 成功实现并模拟了一个计算机的原始模型机，其中包括控制器、运算器、存储器以及 I/O 设备等组件。我对指令设计、模型机整体结构以及控制器的多种实现方式有了更加清晰和 系统的理解，形成了一个完整的概念框图。

通过将指令的执行逻辑拆分为 **“微操作序列”**（如取指的 PC→MAR→MDR→IR，执行的 R0→ALU→R0），并存储在 ROM 中，替代传统硬布线控制的复杂门电路，深刻理解 “微程序是硬件与软件的桥梁”

UPC（微程序计数器）提供地址指针，ROM 存储微程序，μIR 拆分控制信号，微地址形成电路实现分支跳转，四者构成 **“地址生成→微指令读取→信号输出→地址更新”** 的闭环，确保微程序按序执行（如取指周期复用、指令执行分支跳转）。

明确**不同寻址方式**在微操作层面的差异 —— 立即数寻址（MOV R0,#0x55aa）需 PC 读取 RAM 中的立即数，直接寻址（LDR R1,0xA0）需 MAR 加载指令中的固定地址，间接寻址（STR R0,[R2]）需 MAR 加载寄存器中的动态地址，理解 “寻址方式本质是‘操作数地址的生成方式’，由 MAR 的地址来源决定”。

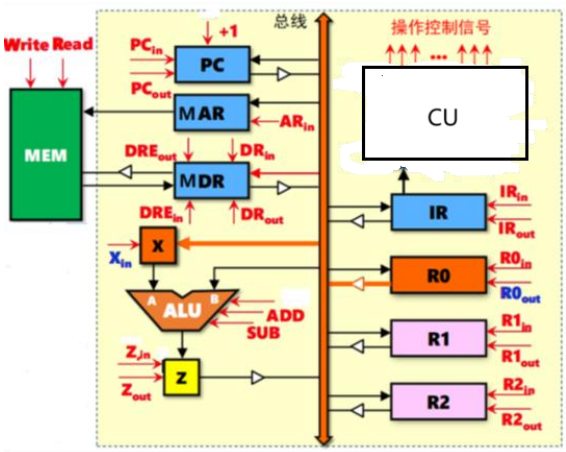
将每条指令拆解为 **“取指周期（固定 4 节拍）+ 执行周期（2~6 节拍）”**，例如 ADD 指令仅需 4 节拍（R0→X→ALU+R1→Z→R0），而 LDR 指令需 6 节拍（PC 读地址→MAR 加载地址→RAM 读数据→R1 写入），体会 “指令复杂度决定执行节拍数，硬件资源（如 ALU、RAM）的占用时间是节拍划分的核心依据”。

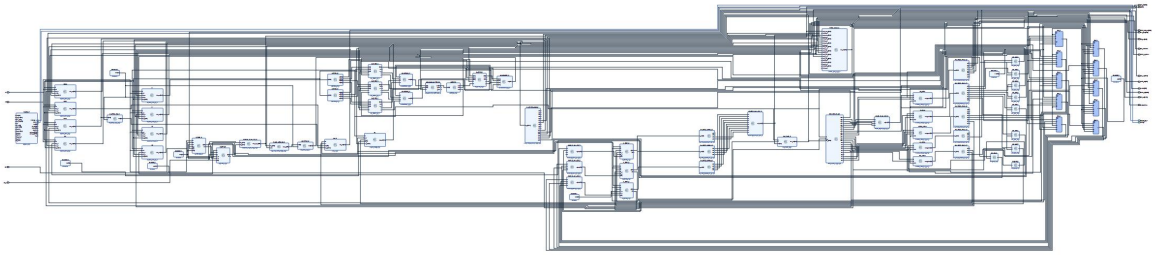
掌握 **ROM 地址分配原则** —— 取指周期微地址固定（0~3），每条指令的执行微地址独立（MOV→16~29，LDR→32~37），预留扩展空间（1024 个单元仅用 30 余个），理解 “微地址分配需兼顾‘顺序执行’与‘分支跳转’，操作码译码结果直接映射执行微地址的首地址”。

模型机采用**单数据总线**连接所有部件（寄存器、ALU、RAM），虽存在 “同一时间仅能传输 1 组数据” 的瓶颈，但简化了硬件设计

6、附录

附录 1：数据通路图





附录 2：微程序流程图

1. 取值周期：

节拍	微地址	微指令	微操作
T0	0	0x00E181	PC->MAR
T1	1	0x000A11	M(MAR)->MDR, PC+1->PC
T2	2	0X00C041	MDR->IR
T3	3	0X000003	QJP

2.各指令的执行周期：

（1）MOV R0,#0x55aa（立即数传送，微地址 16~18、24~25，5 个节拍）

节拍	微地址	微指令	微操作
T4	16	0x00E181	PC→MAR
T5	17	0x000A11	M(MAR)→MDR ，PC+1→PC
T6	18	0x000004	MJP（无条件转移）
T7	24	0x00C081	MDR→R0
T8	25	0x000002	JP（返回取指）

(2) LDR R1,0xA0（直接寻址读，微地址 32~37，6 个节拍）

节拍	微地址	微指令	微操作
T4	32	0x00E181	PC→MAR
T5	33	0x000A11	M(MAR)→MDR ，PC+1→PC
T6	34	0x00C181	MDR→MAR
T7	35	0x000211	M(MAR)→MDR
T8	36	0x00C0C1	MDR→R1
T9	37	0x000002	JP（返回取指）

(3) ADD R0,R1（寄存器加法，微地址 48~51，4 个节拍）

节拍	微地址	微指令	微操作
T4	48	0x004141	R0→X（ALU 输入寄存器）
T5	49	0x246401	X+R1→Z（ALU 输出寄存器）
T6	50	0x00A081	Z→R0
T7	51	0x000002	JP（返回取指）

(4) MOV R2,#0xFA（立即数传送，微地址 16~18、28~29，5 个节拍）

节拍	微地址	微指令	微操作
T4	16	0x00E181	PC→MAR
T5	17	0x000A11	M(MAR)→MDR ，PC+1→PC
T6	18	0x000004	MJP（无条件转移）
T7	28	0x00C101	MDR→R2
T8	29	0x000002	JP（返回取指）

（5）STR R0,[R2]（寄存器间接写，微地址 64~67，4 个节拍）

节拍	微地址	微指令	微操作
T4	64	0x008181	R2→MAR
T5	65	0x0041C1	R0→MDR
T6	66	0x010021	MDR→M(MAR)
T7	67	0x000002	JP（返回取指）

（6）HALT（停机指令，微地址 80，1 个节拍）

节拍	微地址	微指令	微操作
T4	80	0x000008	R2→MARG=1（停机控制信号）

附录 3：微程序

	微操作	微地址	μIR$_{23}$μIR$_{22}$μIR$_{21}$μIR$_{20}$μIR$_{19}$μIR$_{18}$μIR$_{17}$μIR$_{16}$	μIR$_{15}$μIR$_{14}$μIR$_{13}$	μIR$_{12}$μIR$_{11}$μIR$_{10}$	μIR$_{9}$	μIR$_{8}$μIR$_{7}$μIR$_{6}$	μIR$_{5}$μIR$_{4}$μIR$_{3}$	μIR$_{2}$μIR$_{1}$μIR$_{0}$	十六进制代码		
			M S3 S2 S1 S0 C0	DREout	输出Bout分配	PC计数器	Zin	DREin	输入in分配	RAM操作	G	转移方式	CROM		
取指周期	PC->MAR	000	000000	0	111	00	0	0	110	00	0	001	00	E1	81
	M(MAR)->MDR PC+1->PC	001	000000	0	000	01	0	1	000	01	0	001	00	0A	11
	MDR->IR	002	000000	0	110	00	0	0	001	00	0	001	00	C0	41
	OJP	003	000000	0	000	00	0	0	000	00	0	011	00	00	03
1.4立即数寻址	PC->MAR	010	000000	0	111	00	0	0	110	00	0	001	00	E1	81
	M(MAR)->MDR PC+1->PC	011	000000	0	000	01	0	1	000	01	0	001	00	0A	14
	MJP	012	000000	0	000	00	0	0	000	00	0	100	00	00	04
1立即数寻址	MDR->R0	018	000000	0	110	00	0	0	010	00	0	001	00	C0	81
	JP	019	000000	0	000	00	0	0	000	00	0	010	00	00	02
4立即数寻址	MDR->R2	01C	000000	0	110	00	0	0	100	00	0	001	00	C1	01
	JP	01D	000000	0	000	00	0	0	000	00	0	010	00	00	02
2.以0x32为地址访问内存，读取的数据放入R1	PC->MAR	020	000000	0	111	00	0	0	110	00	0	001	00	E1	81
	M(MAR)->MDR PC+1->PC	021	000000	0	000	01	0	1	000	01	0	001	00	0A	11
	MDR->MAR	022	000000	0	110	00	0	0	110	00	0	001	00	C1	81
	M(MAR)->MDR	023	000000	0	000	00	0	1	000	01	0	001	00	02	11
	MDR->R1	024	000000	0	110	00	0	0	011	00	0	001	00	C0	C1
	JP	025	000000	0	000	00	0	0	000	00	0	010	00	00	02
3寄存器寻址	R0->X	030	000000	0	010	00	0	0	101	00	0	001	00	41	41
	X+R1->Z	031	010010	0	011	00	1	0	000	00	0	001	24	64	01
	Z->R0	032	000000	0	101	00	0	0	010	00	0	001	00	A0	81
	JP	033	000000	0	000	00	0	0	000	00	0	010	00	00	02
5把R0的内容保存在内存单元中，内存地址是R2的内容	R2->MAR	040	000000	0	100	00	0	0	110	00	0	001	00	81	81
	R0->MDR	041	000000	0	010	00	0	0	111	00	0	001	00	41	C1
	MDR->M(MAR)	042	000000	1	000	00	0	0	000	10	0	001	01	00	21
	JP	043	000000	0	000	00	0	0	000	00	0	010	00	00	02
6	G=1	050	000000	0	000	00	0	0	000	00	1	000	00	00	08

附录 4：各控制信号的列表

控制信号名称	功能描述	作用部件	信号类型（有效方式）	信号来源
Z_LOAD	控制 ALU 结果寄存器 Z 加载数据（如 ALU 运算结果）	Z[15:0]	时序信号（时钟上升沿有效）	微指令的 Z 寄存器加载控制位 + 系统时钟
IR_LOAD	控制指令寄存器 IR 加载数据（如来自数据总线的指令）	IR[15:0]	时序信号（时钟上升沿有效）	微指令的 IR 寄存器加载控制位 + 系统时钟
R0_WR	控制通用寄存器 R0 写入数据（如总线数据、Z 寄存器结果）	R0[15:0]	时序信号（时钟上升沿有效）	微指令的 R0 写控制位 + 系统时钟
R1_WR	控制通用寄存器 R1 写入数据	R1[15:0]	时序信号（时钟上升沿有效）	微指令的 R1 写控制位 + 系统时钟
R2_WR	控制通用寄存器 R2 写入数据	R2[15:0]	时序信号（时钟上升沿有效）	微指令的 R2 写控制位 + 系统时钟
RST_CTRL	全局复位控制，使 Z/IR/R0-R2 同步清零	所有寄存器（Z/IR/R0-R2）	异步信号（假设高电平有效）	外部 RST 信号

附录 5：各控制信号的逻辑表达式

Z_LOAD = (μIR[5] == 1'b1) & CLK_POS & (~RST);
IR_LOAD = (μIR[4] == 1'b1) & CLK_POS & (~RST);
R0_WR = (μIR[3] == 1'b1) & CLK_POS & (~RST);
R1_WR = (μIR[2] == 1'b1) & CLK_POS & (~RST);
R2_WR = (μIR[1] == 1'b1) & CLK_POS & (~RST);
RST_CTRL = RST;

Cycle	IR	R0	R1	R2	Z	MAR	MDRR	MDRW
1	0000	zzzz	zzzz	zzzz	zzzz	0000	zzzz	zzzz
2	0500	zzzz	zzzz	zzzz	zzzz	0000	0500	zzzz
3	0500	zzzz	zzzz	zzzz	zzzz	0000	zzzz	zzzz
4	0500	zzzz	zzzz	zzzz	zzzz	0001	zzzz	zzzz
5	0500	zzzz	zzzz	zzzz	zzzz	0001	zzzz	zzzz
6	0500	zzzz	zzzz	zzzz	zzzz	0001	zzzz	zzzz
7	0500	zzzz	zzzz	zzzz	zzzz	0001	55aa	zzzz
8	0500	zzzz	zzzz	zzzz	zzzz	0001	zzzz	zzzz
9	0500	zzzz	zzzz	zzzz	zzzz	0002	zzzz	zzzz
10	0500	zzzz	zzzz	zzzz	zzzz	0002	zzzz	zzzz
11	0938	zzzz	zzzz	zzzz	zzzz	0002	0938	zzzz
12	0938	zzzz	zzzz	zzzz	zzzz	0002	zzzz	zzzz
13	0938	zzzz	zzzz	zzzz	zzzz	0003	zzzz	zzzz
14	0938	zzzz	zzzz	zzzz	zzzz	0003	zzzz	zzzz
15	0938	zzzz	zzzz	zzzz	zzzz	00a0	00a0	zzzz
16	0938	zzzz	zzzz	zzzz	zzzz	00a0	zzzz	zzzz
17	0938	zzzz	zzzz	zzzz	zzzz	00a0	zzzz	zzzz
18	0938	zzzz	zzzz	zzzz	zzzz	00a0	aa55	zzzz
19	0938	zzzz	zzzz	zzzz	zzzz	00a0	zzzz	zzzz
20	0938	zzzz	zzzz	zzzz	zzzz	0004	zzzz	zzzz
21	0938	zzzz	zzzz	zzzz	zzzz	0004	zzzz	zzzz
22	1109	zzzz	zzzz	zzzz	zzzz	0004	1109	zzzz
23	1109	zzzz	zzzz	zzzz	zzzz	0004	zzzz	zzzz
24	1109	55aa	zzzz	zzzz	zzzz	0004	zzzz	zzzz
25	1109	zzzz	aa55	zzzz	zzzz	0004	zzzz	zzzz
26	1109	zzzz	zzzz	zzzz	ffff	0004	zzzz	zzzz
27	1109	zzzz	zzzz	zzzz	zzzz	0004	zzzz	zzzz

28	1109	zzzz	zzzz	zzzz	zzzz	0005	zzzz	zzzz
29	1109	zzzz	zzzz	zzzz	zzzz	0005	zzzz	zzzz
30	0540	zzzz	zzzz	zzzz	zzzz	0005	0540	zzzz
31	0540	zzzz	zzzz	zzzz	zzzz	0005	zzzz	zzzz
32	0540	zzzz	zzzz	zzzz	zzzz	0006	zzzz	zzzz
33	0540	zzzz	zzzz	zzzz	zzzz	0006	zzzz	zzzz
34	0540	zzzz	zzzz	zzzz	zzzz	0006	zzzz	zzzz
35	0540	zzzz	zzzz	zzzz	zzzz	0006	00fa	zzzz
36	0540	zzzz	zzzz	zzzz	zzzz	0006	zzzz	zzzz
37	0540	zzzz	zzzz	zzzz	zzzz	0007	zzzz	zzzz
38	0540	zzzz	zzzz	zzzz	zzzz	0007	zzzz	zzzz
39	0d12	zzzz	zzzz	zzzz	zzzz	0007	0d12	zzzz
40	0d12	zzzz	zzzz	zzzz	zzzz	0007	zzzz	zzzz
41	0d12	zzzz	zzzz	00fa	zzzz	00fa	zzzz	zzzz
42	0d12	ffff	zzzz	zzzz	zzzz	00fa	zzzz	zzzz
43	0d12	zzzz	zzzz	zzzz	zzzz	00fa	zzzz	ffff
44	0d12	zzzz	zzzz	zzzz	zzzz	00fa	zzzz	zzzz
45	0d12	zzzz	zzzz	zzzz	zzzz	0008	zzzz	zzzz
46	0d12	zzzz	zzzz	zzzz	zzzz	0008	zzzz	zzzz
47	1400	zzzz	zzzz	zzzz	zzzz	0008	1400	zzzz
48	1400	zzzz	zzzz	zzzz	zzzz	0008	zzzz	zzzz
49	1400	zzzz	zzzz	zzzz	zzzz	0008	zzzz	zzzz
50	1400	zzzz	zzzz	zzzz	zzzz	0008	zzzz	xxxx
51	1400	zzzz	zzzz	zzzz	zzzz	0008	zzzz	xxxx
52	1400	zzzz	zzzz	zzzz	zzzz	0008	zzzz	xxxx
53	1400	zzzz	zzzz	zzzz	zzzz	0008	zzzz	xxxx
54	1400	zzzz	zzzz	zzzz	zzzz	0008	zzzz	xxxx
55	1400	zzzz	zzzz	zzzz	zzzz	0008	zzzz	xxxx
56	1400	zzzz	zzzz	zzzz	zzzz	0008	zzzz	xxxx
57	1400	zzzz	zzzz	zzzz	zzzz	0008	zzzz	xxxx
58	1400	zzzz	zzzz	zzzz	zzzz	0008	zzzz	xxxx
59	1400	zzzz	zzzz	zzzz	zzzz	0008	zzzz	xxxx